

Forma User Guide

Forma is a VS Code extension that converts Markdown into live document previews and PDFs. Open a Markdown file and use the Preview or PDF buttons in the editor title bar, or run commands from the Command Palette.

Japanese Version: [日本語版](#).

Initial Setup

1. Open a VS Code workspace containing your Markdown files.
2. If using project-specific templates, run `Forma: Initialize Project Config` once.
3. Edit the files generated inside the `.forma` directory at the root of the workspace, then open the preview with `Forma: Open Preview`.

This initialization command does not overwrite existing files. If a target file does not exist in the `.forma` directory, the extension's built-in defaults are used.

Exporting PDF

With the Markdown file active, run `Forma: Export PDF`. By default, `<filename>.pdf` is created in the same directory as the target Markdown file. The PDF uses the configurations defined in `.forma/export.scss` and `puppeteer.json`.



PDF export requires Chromium or Google Chrome. If it cannot be detected automatically, specify the executable path in the VS Code setting `forma.puppeteer.puppeteerRunnerPath`. The environment variable `PUPPETEER_EXECUTABLE_PATH` can also be used.

If you want the PDF to open automatically after export, set `forma.common.useAutoOpenPDF` to `true`. The default is `false`.

Project Files

As described in [Initial Setup](#), running `Forma: Initialize Project Config` creates a `.forma` directory directly under the workspace root. This is the Forma configuration directory dedicated to that workspace. You can place the following files here:

- `default.scss`: Standard preview theme
- `print.scss`: Print preview theme applied when selected via `theme: print`
- `export.scss`: Dedicated theme used during PDF export
- `parser.js`: Defines hooks for transforming Markdown and HTML
- `puppeteer.json`: Settings for paper size, margins, browser launch arguments, destination, etc.

By defining `forma.theme.libraries`, `forma.parser.libraries`, and `forma.puppeteer.libraries` in your VS Code settings, you can use named libraries. For example:

```
// VSCode Configuration
```

Note: `@` is an alias representing the workspace root directory.

With this configured, you can select and use it from the Markdown front matter like this:

```
---
```

You can also change the names of configuration files to load by modifying `forma.theme.entry`, `forma.parser.entry`, and `forma.puppeteer.entry`.

Front Matter

In default Forma, the following front matter properties are available:

- `theme` : Specifies the named theme to use
- `parser` : Specifies the named parser to use
- `puppeteer` : Specifies the named Puppeteer configuration to use
- `numbering` : Adds numbers to the beginning of specified headings
- `header` : Defines document header content
- `footer` : Defines document footer content

These front matter properties are used as follows:

```
---
```

Details on Numbering

`numbering` accepts a `number` or `hyphen-separated numbers` . For example, specifying `1` prefixes level 1 headings (`# Heading`) with a number, displaying them as `# 1. Heading` . Numbers are sequential starting from 1 and are applied to all headings at the same level; however, any heading immediately following `<!-- IGNORE-NUM -->` is excluded from the sequential count.

Details on Headers and Footers

Specification Method

Specify 2 or 3 strings for `header` and `footer` . When 2 strings are provided, they appear on the left and right; when 3 are provided, they appear on the left, center, and right. The following special variables can also be used:

- `title` : Displays the first level 1 heading in the page
- `date` : Displays the date of printing or previewing in `YYYY-MM-DD` format
- `page` : Displays the page number

Styling

Headers and footers can be styled in two places.

First is `defaultHeaderFooterFont` inside `puppeteer.json` . Here, you can specify shared properties for both headers and footers, such as `size` (font size) and `color` (text color). For example:

```
// puppeteer.json
```

With this setting, the font size of the header and footer becomes 12px, and the text color becomes red.

Second is directly within the front matter:

```
---
```

With this configuration, only "Bottom-right footer" will be styled in red with a 12px font size.

Document Directives

The following directives can be used within Markdown:

- `<!-- TOC -->` : Creates a table of contents from level 1 to 6.
- `<!-- TOC[2] -->` : Restricts the table of contents to levels 1 through 2.
- `<!-- TOC[2,4] -->` : Includes levels 2 through 4.
- `<!-- IGNORE-TOC -->` : Excludes the next heading from the table of contents.
- `<!-- IGNORE-NUM -->` : Excludes the next heading from numbering.
- `<!-- ID[custom-id] -->` : Sets an alias on the next heading for internal linking.
- A line containing only `---` : Inserts a page break.

HTML, footnotes, task lists, image sizing like `![Alt](image.png =320x200)`, containers, autolinks, and Prism syntax highlighting for major programming languages and configuration formats are supported.

Web and scripting languages

- `typescript`
- `json`
- `yaml`
- `markdown`
- `bash`
- `shell-session`

Backend and systems languages

- `python`
- `php`
- `java`
- `c`
- `cpp`
- `csharp`
- `go`
- `rust`
- `ruby`
- `kotlin`
- `swift`

Database and configuration formats

- `sql`
- `graphql`
- `docker`
- `ini`
- `toml`

Images and Output Destinations

Images with relative paths are resolved relative to the Markdown file. Paths beginning with `@/` are resolved relative to the workspace root. In `puppeteer.json`, `destination: "."` outputs the PDF to the same directory as the source file, `destination: "@"` outputs to the workspace root, and `destination: "@/exports"` outputs to an `exports` directory beneath it.